

 Agentic Postgres: The first database built for agents 



Product

Developer Hub

Try for free



 Search in Blogs

Category: All posts

Tutorials

How to Migrate Your Data to Timescale (3 Ways)

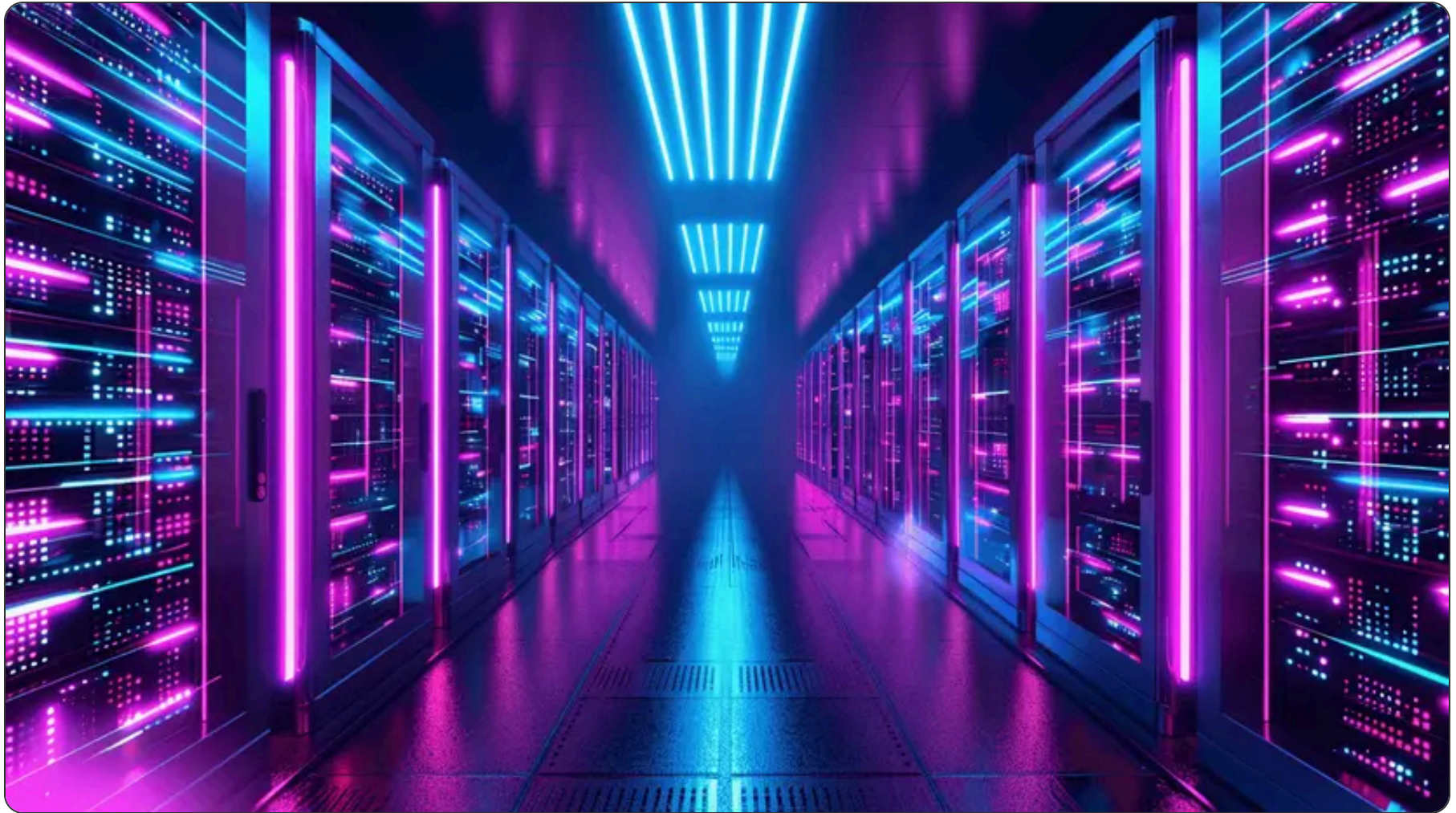


Table of Contents



- 01 Pre-Migration Considerations
- 02 1. Migrate Data Using CSV Files
- 03 2. Pg_dump/pg_restore
- 04 3. Live migration

05 Migrate Your Data Today

Data migration involves careful planning and follow-up tasks such as making a strategy, backing up data, doing quality checks, and ensuring the moved data is correct. When moving data to Timescale, you'll have to follow a few steps, from selecting the compatible data format and tool for migration to focusing on measuring the data transfer, along with testing and ensuring it's all done right.

In this article, we'll show you three ways to migrate your data into our PostgreSQL-but-faster database.



Want to learn more about how to build your application using PostgreSQL/Timescale? Be sure to check these articles—we're taking you from data migration to monitoring your database.

- > [How to Migrate Your Data to Timescale \(3 Ways\)](#)
- > [Building Python Apps With PostgreSQL and psycopg3](#)
- > [Bonus: Psycopg2 vs Psycopg3 Performance Benchmark](#)
- > [Data Visualization in PostgreSQL With Apache Superset](#)
- > [5 Ways to Monitor Your PostgreSQL Database](#)

Pre-Migration Considerations

- Check the size of data in your source database, as larger databases will require more effort, resources, and careful management to migrate. It's also important to understand how the data is distributed across various tables, workload patterns, and downtime requirements, as this will influence the plan and timing for the migration. Additionally, check for archived or static data, as it might need a different approach compared to the frequently accessed data.
- Ensure that the extensions you're using in your current source database are compatible with Timescale. You can check the list of compatible extensions here: [Timescale Extensions Compatibility](#).
- If you're working with multiple databases (in a single PostgreSQL instance), you'll need to choose one of two options for migrating to Timescale. Either create separate Timescale instances for each database or merge your databases into separate schemas in a single Timescale database, which is named **tsdb**.
- In Timescale, the user **tsdbadmin** has the highest access level, but this role doesn't have the same capabilities as a PostgreSQL superuser. Before migrating, make sure to verify whether your application uses any superuser privileges and plan for alternative solutions if needed.
- When migrating from a self-hosted TimescaleDB (source) to another Timescale instance (target), it's required to have the same Timescale versions on both sides. For upgrading your TimescaleDB extension, refer to the [Upgrade TimescaleDB](#).

To check your Timescale version, run the following command:

```
SELECT extversion FROM pg_extension WHERE extname='timescaledb';
```

- When moving sensitive data, it's important to keep it safe for compliance requirements. This can be done by sending the data through proper encrypted channels.
- Make sure you have enough hardware resources such as storage, network bandwidth, and processing power to handle the migration without running into problems like running out of space or the system being too slow. If you're moving more than 400 GB of data, open a support request to ensure that enough space is available for your Timescale instance.
- Since Timescale instances operate on Amazon's cloud, it is recommended to use an AWS EC2 instance located in the same region as your Timescale instance for migration purposes. This setup will help with reducing network delays and improve data transfer speed.

1. Migrate Data Using CSV Files

CSV is a common format for gathering and storing data in a tabular format structure. Each line in a CSV file usually represents a single record or row of a table. The migration of data using CSV files is a straightforward process across various data systems. PostgreSQL provides functionality to both import and export CSV files.

In PostgreSQL and Timescale, the **COPY** command helps to move data in and out of CSV files. It provides different options such as custom separators/delimiters, error handling, and null value setting. COPY works faster than inserts because it reduces overheads like connection overhead, network round trips, and managing transactions. This makes it a really good candidate for bulk data ingestion.

Create a CSV file from a table using COPY TO:

This command exports the data from `schema.table` into a CSV file with a header row.

```
COPY schema.table TO '/DIR/schema_table.csv' DELIMITER ',' CSV HEADER;
```

For a specific result set:

```
COPY (Select * from schema.table where ....) TO '/DIR/schema_table.csv' DELIMITER ',' CSV HEADER;
```

Load the CSV file into a table using COPY FROM:

Make sure that the table is created in the target database inside the same schema as the source.

```
COPY schema.table FROM '/DIR/schema_table.csv' DELIMITER ',' CSV HEADER;
```

Load the CSV file into Timescale using Timescaledb-parallel-copy:

The Timescale Parallel Copy tool, developed in Go, is used to efficiently import large CSV data files into a Timescale hypertable.

Before using this tool, check that the time column inside the table uses the TIMESTAMPTZ datatype and ensure that the target table has been converted into a hypertable.

To load a CSV file, use this command:

```
timescaledb-parallel-copy --connection "host=<CLOUD_HOST> user=tsdbadmin password=<CLOUD_PASSWORD> port=
<CLOUD_PORT>
sslmode=require" --db-name tsdb --table <schema.table> --file <schema_table>.csv --workers 4 --copy-options
"CSV" --reporting-period 30s --batch-size 10000
```

The TimescaleDB Parallel Copy tool is great for importing data faster because it uses many workers at once, unlike the usual COPY commands that use just one thread. It's also fine-tuned for working with hypertables, which are designed for time-series data.

Check it out here: [Timescale Parallel Copy Tool](#).

Considerations:

- COPY TO requires read privileges, and COPY FROM requires write privileges on a table.
- COPY TO can be used only with plain tables, not views, and does not copy rows from child tables or child partitions.
- For bulk loading using CSV files using COPY, adjust GUC (grand unified configuration) parameters on the target database, such as **max_wal_size** and **checkpoint_timeout**, for better performance.
- It is often recommended to create indexes and constraints after bulk load operation for speedup.

- ANALYZE your tables after bulk load operations for correct statistics.
- When using **timescaledb-parallel-copy**, monitor your system's resource usage, as it can use more CPU and memory, and then adjust the number of worker threads accordingly.
- CSV files can also be imported or exported using pgAdmin.

2. Pg_dump/pg_restore

Migrating data using pg_dump and pg_restore is a common and efficient method, especially suited for database sizes less than 100 GB.

The most common tool used for creating logical backups (a sequence of SQL statements) in PostgreSQL is the **pg_dump/pg_dumpall** utility. **pg_dump** runs **SELECT** in the background and takes **access shared** locks on database objects, which means it doesn't block reads/writes inside the table.



Need a guide on pg_restore?

Take a backup of roles inside the database first (only necessary if you're using roles other than the default Postgres role in your database):

```
pg_dumpall -d "<Source_Connection_String>" --quote-all-identifiers --roles-only -f roles.sql
```

Where

Source_Connection_String can be specified as:

```
postgres://<sourcedbuser>:<sourcepassword>@<sourcedbhost>:<sourcedbport>/<sourcedbname>
```

Target_Connection_String

```
postgres://tsdbadmin:<targetpassword>@<targetdbhost>:<targetdbport>/tsdb
```

Run the following command to take a backup using pg_dump in plain format:

```
pg_dump -d <Connection_String> --quote-all-identifiers --no-tablespaces --no-owner --no-privileges **--  
format=plain** -f backup.sql
```

For backups using parallel workers, use the directory format:

```
pg_dump -d <Source_Connection_String> --quote-all-identifiers --no-tablespaces --no-owner --no-privileges **--  
format=directory** --jobs=8 -f backup.dir
```

- **--no-tablespaces** is used because Timescale does not support tablespaces other than the default.
- **--no-owner** is used because Timescale's tsdbadmin cannot assign ownership in all cases. This flag means that everything is owned by the user used to connect to the target, regardless of ownership in the source.
- **--no-privileges** is used because Timescale's tsdbadmin user is not a superuser and cannot assign privileges in all cases. This flag means that privileges assigned to other users must be reassigned in the target database as a manual clean-up task.
- **--jobs** is used for parallel workers.
- **-f** is used for specifying the backup filename.

Restore with no concurrency

Since the backup is in plain format, **psql** can be used to import the dump into the target PostgreSQL database.

```
Psql -h Target_host -p Target_port -U tsdbadmin -d tsdb -f backupfile.sql
```

If background jobs are running on Timescale (target database), the better way to restore is the following:

```
psql -h Target_host -p Target_port -U tsdbadmin -d tsdb -v ON_ERROR_STOP=1 --echo-errors -f roles.sql -c "SELE  
public.timescaledb_pre_restore();" -f backupfile.sql -c "SELECT public.timescaledb_post_restore();"
```

Where

`timescaledb_pre_restore()` is called before restoration and stops background workers.

`timescaledb_post_restore()` is called after restoration and restarts background workers.

Restore with concurrency

`_timescaledb_catalog` schema doesn't support concurrent restoration using `pg_restore`. To avoid errors, first restore the `timescaledb_catalog` schema, and then you can load the rest of the database concurrently.

```
pg_restore -d "Target_Connection_String" --format=directory --schema='_timescaledb_catalog' --exit-on-error --  
no-tablespaces --no-owner --no-privileges backupfile.dir
```

Next, restore everything except the `_timescaledb_catalog` using `--exclude-schema` and `--jobs` flag:

```
pg_restore -d "Target_Connection_String" --format=directory --jobs=8 --exclude-schema='_timescaledb_catalog'  
exit-on-error --disable-triggers --no-tablespaces --no-owner --no-privileges backupfile.dir
```

Considerations:

- pg_dump operates using a single thread when creating backups in Tar, Custom, and Plain format. However, it supports parallel backup only when backing up in the directory format.
- Be aware that pg_dump can conflict with DDL (Data Definition Language) commands like dropping/altering a table or TRUNCATE, as these operations require an exclusive lock on the table.
- When using self-hosted TimescaleDB, make sure to stop retention, compression, and continuous aggregate refresh policies, as they also require **ACCESS EXCLUSIVE** locks. To prevent issues, run the following query to stop background jobs before you start pg_dump.

```
SELECT public.alter_job(id::integer, scheduled=>false)  
FROM _timescaledb_config.bgw_job  
WHERE id >= 1000;
```

Once pg_dump is completed, you can enable the job with the following query:

```
SELECT public.alter_job(id::integer, scheduled=>true)  
FROM _timescaledb_config.bgw_job
```

```
WHERE id >= 1000;
```

- To speed up the restore process, you can tweak settings such as **max_wal_size**, **checkpoint_timeout**, and **wal_buffers** in Timescale (located under Services -> Choose your Database instance -> Operations -> Database Parameters). Adjusting these parameters can reduce the I/O load on your system.
- When moving data to a newer major version using pg_dump/pg_restore, it is advised to use the binaries from the higher PostgreSQL version.

Check out our docs for more troubleshooting on migrating your database.



Editor's Note: Pg_dump and pg_restore can also be done using pgAdmin.

3. Live migration

When you need to move a large amount of data with minimal downtime, live migration is a great strategy provided by Timescale. It uses a tool called **pgcopydb** in the background, which works in two main steps:

- **Automated data transfer:** pgcopydb automates the process of transferring data between two running PostgreSQL servers using pg_dump in directory format and pg_restore. Normally, you cannot run dump and restore on the fly with the --jobs

option because `pg_dump` writes to local directory files first. But with `pgcopydb`, you can quickly copy a database to another server.

- **Capturing and applying changes:** It also captures ongoing changes from the source database and applies them to the target database using PostgreSQL's logical decoding and the `wal2json` plugin. This keeps the target database up-to-date with the source during the migration.

For more information, check the `pgcopydb` documentation.

How to start live migration

- Set **wal_level** to logical on the source database. You can do this by altering the system settings:

```
ALTER SYSTEM SET wal_level = logical;
```

- Set `old_snapshot_threshold` to -1:

```
ALTER SYSTEM set old_snapshot_threshold=-1
```

If you're using a managed service (AWS RDS, Azure Flexible Server, etc.), adjust these settings through your cloud provider's interface.

- Restart the database for these changes to take effect.
- Make sure your tables and hypertables on the source have a primary key or a replica identity for replicating UPDATE and DELETE commands. The key will be used to identify rows on the target side. If a table doesn't have a primary key, set a replica identity using an index. **The index must be unique, not partial, not deferrable, and include only columns marked NOT NULL.**

```
ALTER TABLE {table_name} REPLICA IDENTITY USING INDEX {_index_name};
```

For tables without a primary key or unique index, use **REPLICA IDENTITY FULL**, though this can slow down replication for tables with frequent updates/deletes. The issue with **REPLICA IDENTITY FULL** is that for each UPDATE or DELETE statement, PostgreSQL will log the old column values inside write-ahead-log (WAL) files, which can result in replication overhead and can incur I/O load on the source database.

- Set your database URIs as environment variables on your EC2 instance:

```
export SOURCE=postgres://{user}:{password}@{source host}:{source port}/{dbname}
```

```
export TARGET=postgres://{tsdbadmin}:{password}@{target host}:{target port}/tsdb
```

- Download and run the live migration Docker image:

```
docker run --rm -dit --name live-migration -e PGCOPYDB_SOURCE_PGURI=$SOURCE -e PGCOPYDB_TARGET_PGURI=$TARGET -  
~/live-migration:/opt/timescale/ts_cdc timescale/live-migration:v0.0.1
```

This command will perform the following steps:

- Take a snapshot of the source database and migrate the schema (table structure) to the target database (tsdb).
- If you're migrating data from PostgreSQL, it will prompt you to create hypertables in the target database. Once you have finished creating hypertables, you need to press "**c**" or "**Enter**" to proceed. You can do it by pressing the **c** key. If you're migrating from self-hosted TimescaleDB, it won't show this prompt.
- Migrate existing data from source to target (using pg_dump/pg_restore in the background with directory format). The duration of this process will depend on the size of your database.
- During the initial data copy, it will also capture new changes in local JSON and SQL files.
- Once the initial data sync is done, it will start streaming changes created during the initial data sync as per commit order (by parsing JSON files). During this process, it will also display the lag between the source and target databases in terms of WAL offset size.

[WATCH] Source DB - Target DB => **128MB** Keep an eye on the **data** transfer progress and **replication** lag.

- When the difference in replication lag between your source and the target is under 30 MBs, the system will run ANALYZE on all tables on target for updating statistics. This is required for optimal query performance.
- For the final cutover, stop your application and monitor replication. You can also use the **pg_stat_replication** view to track this. Once replication lag is 0, it will prompt you the following on screen:

```
[WATCH] Source DB - Target DB => 0MB. Press "c" (and ENTER) to proceed  
      Syncing last LSN in Source DB to Target DB ...
```

After this, the live migration tool will handle migrating sequences and cleaning the temporary files created during migration. If everything goes as planned, you'll get a final message, **Migration successfully completed**, which indicates that your data has been successfully moved. You can now connect your application to a Timescale database.



Moving from AWS RDS for PostgreSQL? Here's a step-by-step guide to assist you along the way.

Considerations:

- Tables with heavy traffic should have primary keys and minimal UPDATE or DELETE operations.
- If your source TimescaleDB version is older than v2.12, make sure your hypertables have primary keys, as they don't support REPLICA IDENTITY functionality. If you cannot create primary keys, you'll need to update your TimescaleDB to a newer version (2.12 or above).

Migrate Your Data Today

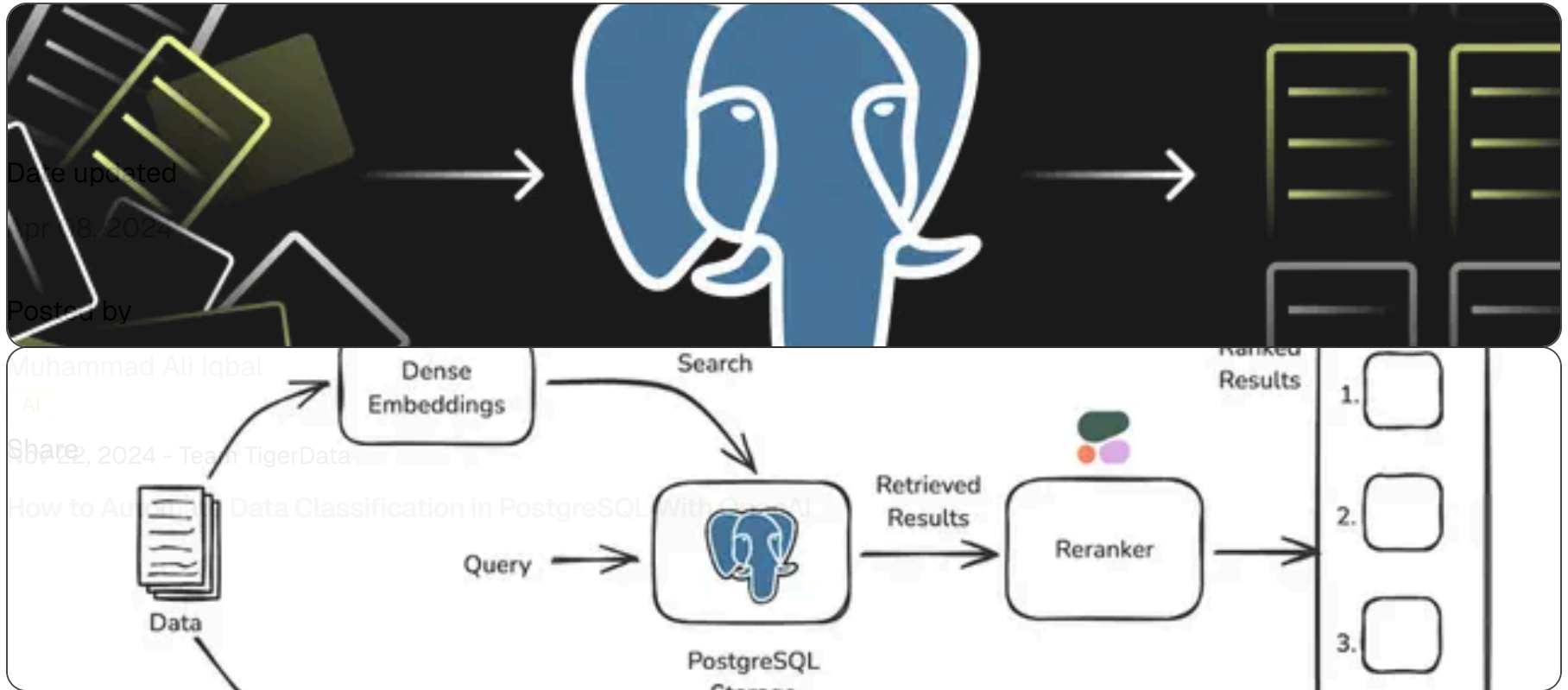
If you're browsing this blog post to assess how easy it would be to migrate to Timescale, we hope this step-by-step guide will clear your doubts and help you choose the best way to move your data. Remember, you can always check our migration docs for more details.

For added peace of mind, know that when migrating to our mature cloud platform, you will have access to our expert Support team to help you along the way from day one—even as a trialer. Sign up today; it's free for 30 days, no credit card required.

Next steps

 **Learn how to build Python apps using PostgreSQL/Timescale.**

Related posts



AI

Nov 15, 2024 - Team TigerData

Products

- Time Series and Analytics
- AI and Vector
- Enterprise Plan
- Cloud Status
- Support
- Security
- Cloud Terms of Service

Learn

- Documentation
- Blog
- Forum
- Tutorials
- Changelog
- Success Stories
- Time Series Database

Company

- Contact Us
- Careers
- About
- Brand
- Community
- Code Of Conduct
- Events



Subscribe to the Tiger Data Newsletter

Your email

Subscribe

By submitting, you acknowledge Tiger Data's Privacy Policy

2025 (c) Timescale, Inc., d/b/a Tiger Data. All rights reserved.

[Privacy preferences](#)

[Legal](#)

[Privacy](#)

[Sitemap](#)